Lab 30 — Automation & Orchestration (Mini-SOAR)

You will write a small bash playbook that watches `/var/log/auth.log`, auto-bans the source IP after N failures, opens a "ticket" file, and logs to syslog. This is the SOAR pattern in 50 lines.

Run on https://killercoda.com/playgrounds/scenario/ubuntu

Required software (free): - `bash`, `awk`, `inotify-tools`, `iptables`

Step 1 — Install

```
apt update && apt install -y inotify-tools
```

Step 2 — Write the playbook

```
cat > /usr/local/bin/soar.sh <<'EOF'
#!/bin/bash
# Mini-SOAR: auto-ban brute-force IPs after 5 failures within 60s
THRESHOLD=5
```

```
WINDOW=60

TICKETS=/var/tickets

LOG=/var/log/soar.log

mkdir -p "$TICKETS"

declare -A counter

declare -A first_seen

tail -F /var/log/auth.log | while read -r line; do

  ip=$(grep -oE 'from [0-9.]+' <<<"$line" | awk '{print $2}')
```

```
  case "$line" in

    *"Failed password"*|*"Invalid user"*)

      [ -z "$ip" ] && continue

      now=$(date +%s)

      [ -z "${first_seen[$ip]}" ] && first_seen[$ip]=$now

      counter[$ip]=$(( ${counter[$ip]:-0} + 1 ))

      elapsed=$(( now - first_seen[$ip] ))

      if [ $elapsed -gt $WINDOW ]; then

        counter[$ip]=1; first_seen[$ip]=$now

      fi
```

```
      if [ ${counter[$ip]} -ge $THRESHOLD ]; then

        if ! iptables -C INPUT -s $ip -j DROP 2>/dev/null; then

          iptables -I INPUT -s $ip -j DROP

          ticket="$TICKETS/$(date +%s)-$ip.json"

          cat > "$ticket" <<JSON

{"ts":"$(date -u +
%FT%TZ)","ip":"$ip","action":"banned","reason":"ssh-brute","count":$
{counter[$ip]}}

JSON

          logger -t soar "BANNED $ip (count=${counter[$ip]})
ticket=$ticket"

          echo "[$(date)] BANNED $ip" >> $LOG

        fi
```

```
      fi


      ;;


  esac


done


EOF


chmod +x /usr/local/bin/soar.sh
```

Step 3 — Run it as a unit

```
cat > /etc/systemd/system/soar.service <<'EOF'


[Unit]


Description=Mini SOAR
```

```
After=network.target

[Service]

ExecStart=/usr/local/bin/soar.sh

Restart=always

[Install]

WantedBy=multi-user.target

EOF

systemctl daemon-reload
```

```
systemctl enable --now soar.service
```

Step 4 — Trigger and observe

```
for i in $(seq 1 6); do ssh -o PubkeyAuthentication=no -o
StrictHostKeyChecking=no \
   bad@127.0.0.1 2>/dev/null; done
sleep 3
ls /var/tickets/
iptables -L INPUT -n | head
journalctl -t soar -n 5 --no-pager
```

You have just demonstrated all five SOAR primitives: 1. **Ingest** (tail auth.log) 2. **Detect** (pattern match) 3. **Decide** (threshold) 4. **Act** (iptables block) 5. **Ticket + Log** (file + syslog)

Step 5 — Add an API call (escalation)

```
# placeholder - in real life: curl -X POST
https://hooks.slack.com/services/... -d "..."
```

Free online tools

- **TheHive (open SOAR / case management)** — https://thehive-project.org
- **Shuffle** (open-source SOAR) — https://shuffler.io
- **n8n** (workflow automation) — https://n8n.io
- **Tines free community edition** — https://www.tines.com/community
- **MISP** (threat intel platform) — https://www.misp-project.org

---

What you learned

- SOAR is just *event* → *decision* → *action* → *record* — automatable in bash.
- The three operational benefits: reaction time, workforce multiplier, audit trail.
- Why API integration (the 5th primitive) is what turns SOAR into a force multiplier.